NutriVision – Requirements Gathering

Project Documentation – Section 3

1. Stakeholder Analysis

Stakeholder	Needs / Expectations
End Users	Simple meal logging, understandable nutritional information, personalized recommendations, privacy and secure storage.
Project Supervisor / Lecturer	Clear objectives, strong documentation, technical implementation, testing and evidence of project planning.
Development Team (Ahmed, Kamel, Osama, Mahmoud, Bassam)	Clear requirements, architecture, APIs, backend services, development tools and testing strategy.

2. User Stories

ID / Title	User Story
US-01 – Record Meal	As a user, I want to enter a description of my meal so that NutriVision can analyze it.
US-02 – Nutritional Advice	As a user, I want to receive nutritional advice about my meal so that I can understand whether it is a healthy choice.
US-03 – Dietary Restrictions	As a user, I want to specify my dietary restrictions so that recommendations can be customized.
US-04 – Recipe Alternatives	As a user, I want to receive healthier recipe alternatives so that I can improve my meals.
US-05 – Meal History	As a user, I want to view previous meals so that I can track my dietary habits.

US-06 – Synchronization	As a user, I want my meal history to synchronize across devices so that I can access it from different devices.
US-07 – Nutritional Dashboard	As a user, I want to see nutritional information in charts so that I can understand my dietary patterns more easily.

3. Use Cases

UC-01 – Analyze Meal

Actor: User
Precondition: The user has entered a meal description.
Main Flow:
1. User opens NutriVision.
2. User enters a meal description.
3. User submits the meal.
4. Application validates the input.
5. Application sends the request to the AI service.
6. AI analyzes the meal.
7. Application displays nutritional recommendations.
8. User can save the meal.

Postcondition: The meal analysis is displayed and can be stored in the user's history.

UC-02 – Generate Alternative Recipe

Actor: User
Precondition: Meal information is available.
1. User selects a meal.
2. User requests an alternative recipe.
3. The user's dietary restrictions are included in the AI prompt.
4. AI generates a healthier alternative.
5. Application displays the recipe.

UC-03 – Synchronize Meal History

Actor: User
1. User saves a meal.
2. Application stores the meal through its data layer.
3. Data is synchronized with Firebase Cloud Firestore.
4. User accesses the application from another device.
5. Application retrieves the synchronized meal data.

4. Functional Requirements

- FR-01: The system shall allow users to enter meal descriptions.
- FR-02: The system shall analyze meal descriptions using an AI service.
- FR-03: The system shall generate nutritional recommendations.
- FR-04: The system shall generate customized recipes.
- FR-05: The system shall consider dietary restrictions when generating recommendations.
- FR-06: The system shall suggest healthier alternatives to meals or snacks.
- FR-07: The system shall allow users to save meals.
- FR-08: The system shall display meal history.
- FR-09: The system shall display nutritional information using charts.
- FR-10: The system shall synchronize meal information using Firebase Cloud Firestore.
- FR-11: The system shall handle API errors and connection failures.
- FR-12: The system shall support secure API request handling using Dio Interceptors where appropriate.
- FR-13: The system shall provide responsive UI layouts.
- FR-14: The system shall validate user input.
- FR-15: The system shall provide appropriate loading and error states.

5. Non-Functional Requirements

Category	Requirement
Performance	The UI should remain responsive and unnecessary widget rebuilds should be minimized.
Performance	API operations should provide suitable loading indicators and controlled timeouts.
Security	API credentials should not be hardcoded into publicly exposed source code.
Security	External communication should use HTTPS.
Security	Firebase Security Rules should restrict unauthorized access.
Usability	The interface should be simple, intuitive and easy to navigate.

Usability	Nutritional information should be presented in understandable formats.
Reliability	Network failures should be handled gracefully without crashing the application.
Reliability	Synchronization failures should produce controlled error states.
Maintainability	Business logic should be separated from UI components.
Maintainability	The codebase should follow modular organization and consistent naming.
Scalability	The architecture should allow future features such as barcode scanning, image recognition and wearable integration.